UNIVERSIDAD DE LA SALLE

**Nombre de la materia**

Calidad del software II.

**Nombre del programa**

Ingeniería de software

**Nombre del estudiante**

MATÍAS VIRGÜEZ

**Nombre de la actividad**

Sesión de triage + plan de mejora.

**Unidad #**

Unidad 3 Actividad 5

**Nombre del profesor**

JEFERSON URREGO GUERRERO.

**Fecha**

31-10-2026

Matías Virgüez

Triage de defectos, análisis causa-raíz y quality gates en CI/CD

Prueba de rendimiento QuickPizza con k6 · Calidad en Software II · Unidad 2

Autor: Matías Virgüez · Fecha: 31 de octubre de 2026 · Trabajo individual

Repositorio: https://github.com/Matiasv18/prueba-rendimiento-quickpizza

## 1. Introducción

Este documento parte de la prueba de rendimiento que hice sobre QuickPizza con k6 (carga, estrés y resistencia). Como trabajé solo, en la sesión de triage asumí los tres roles: moderador, evaluador técnico y secretario. Cada decisión queda escrita con su razón para que se pueda revisar.

Lo primero fue mirar los resultados. Se procesaron más de 22.000 peticiones, hubo 0 % de errores y la latencia quedó dentro del SLO. Entonces, no hay defectos críticos en el sentido de que el usuario no pueda usar el sistema, y por eso no voy a inflar la severidad. Lo que sí encontré son hallazgos con evidencia que incumplen un SLO o que le quitan confiabilidad a la prueba. Esos son los que trato como defectos.

| Indicador | SLO | Carga (20 VUs) | Estrés (hasta 100 VUs) | Resistencia (15 VUs) |
|---|---|---|---|---|
| P95 GET / | < 800 ms | 104.58 ms (cumple) | 101.17 ms (cumple) | 102.29 ms (cumple) |
| P95 POST /api/pizza | < 2000 ms | 659.59 ms (cumple) | 672.54 ms (cumple) | 686.84 ms (cumple) |
| Tasa de error | < 1 % | 0 % (cumple) | 0 % (cumple) | 0 % (cumple) |
| Throughput | ≥ 15 req/s | 7.60 req/s (no cumple) | 18.43 req/s (cumple) | 6.13 req/s (no cumple) |

## 2. Acta de triage

### 2.1 Datos de la sesión

| Campo | Detalle |
|---|---|
| Fecha | 31 de octubre de 2026 |
| Modalidad | Individual (simulación de comité de triage) |
| Roles | Moderador, evaluador técnico y secretario, asumidos por Matías Virgüez |
| Insumo | Resultados crudos de k6 (CSV/JSON), scripts, informe técnico y dashboard del repositorio |
| Referencias | ISTQB Advanced Test Manager v3.0 (defect management) y postmortems sin culpa de |

Matías Virgüez

| Campo | Detalle |
|---|---|
| | Google SRE |

## 2.2 Criterios usados

La severidad dice cuánto daña el defecto. La prioridad dice qué tan pronto hay que actuar. Son cosas distintas: un defecto muy severo en algo que casi nadie usa puede esperar, y uno moderado que bloquea todo el trabajo no.

| Nivel | Severidad (impacto) | Prioridad (urgencia) |
|---|---|---|
| Crítica / P1 | El usuario no puede usar el servicio, o hay pérdida de datos o un riesgo de seguridad | P1: se corrige antes de la próxima ejecución o entrega |
| Alta / P2 | Incumple un SLO o un criterio de aceptación, o deja sin control un riesgo importante | P2: se corrige en la semana siguiente |
| Media / P3 | Afecta la calidad o la confianza, pero cumple el SLO | P3: se corrige cuando haya espacio |
| Baja | Detalle cosmético o de documentación | Sin urgencia |

## 2.3 Fichas de defectos

### DEF-01. Throughput por debajo del SLO en carga y resistencia

- Descripción: el SLO pide al menos 15 req/s, pero la prueba de carga logró 7.60 req/s y la de resistencia 6.13 req/s.
- Pasos para reproducir: (1) ejecutar `k6 run scripts/carga.js`; (2) dividir el total de peticiones entre la duración de la prueba; (3) comparar con el SLO de 15 req/s; (4) repetir con `scripts/resistencia.js`.
- Evidencia: `results/carga.csv` (1.370 iteraciones de 2 peticiones en 6 min = 7.6 req/s), `results/resistencia.csv` (5.539 iteraciones en 30 min = 6.15 req/s), `dashboard/resultados_pruebas.csv` e `report/Informe_tecnico_prueba_rendimiento.pdf`.
- Severidad propuesta: Alta. Prioridad inicial: P1.

### DEF-02. Umbrales de estrés demasiado flojos y punto de quiebre no encontrado

- Descripción: en `estres.js` los umbrales son P95 < 1500 ms (home), P95 < 3000 ms (pizza) y error < 5 %, mientras que el SLO dice 800 ms, 2000 ms y 1 %. Además, con 100 VUs el sistema siguió respondiendo igual de bien, así que nunca se encontró el límite.
- Pasos para reproducir: (1) abrir `scripts/estres.js` y leer la sección `thresholds`; (2) ejecutar `k6 run scripts/estres.js`; (3) revisar P95 y tasa de error cuando se llega a 100 VUs.
- Evidencia: `scripts/estres.js`, `results/estres.csv` (máximo de POST 857 ms, 0 errores en 4.436 iteraciones), captura `report/screenshots/cpu_estres_pico.png`.

- Severidad propuesta: Media. Prioridad inicial: P3.

### DEF-03. Cola larga de latencia en POST /api/pizza

- Descripción: la mediana del POST es ~202 ms, pero el P90 es ~609 ms. Cerca de 1 de cada 5 peticiones supera los 500 ms, y esto pasa igual con 15, 20 o 100 VUs.
- Pasos para reproducir: (1) ejecutar scripts/carga.js; (2) calcular P50, P90 y P95 del POST; (3) contar las peticiones mayores a 500 ms; (4) repetir con scripts/estres.js y comparar.
- Evidencia: results/carga.csv (269 de 1.370 peticiones > 500 ms, 19.6 %), results/estres.csv (910 de 4.436, 20.5 %), results/resistencia.csv (1.172 de 5.539, 21.2 %). El histograma del POST en carga tiene dos grupos: uno entre 100 y 200 ms (673 peticiones) y otro entre 500 y 700 ms.
- Severidad propuesta: Media. Prioridad inicial: P3.

## 2.4 Decisiones del triage

| ID | Severidad (propuesta → acordada) | Prioridad (inicial → acordada) | Responsable | Fecha límite | Justificación |
|---|---|---|---|---|---|
| DEF-01 | Alta → Alta | P1 → P1 | M. Virgüez | 5-nov-2026 | Es el único SLO incumplido, así que la prueba no puede darse por aprobada. La causa ya se conoce y el arreglo es barato, por eso va primero. |
| DEF-02 | Media → Alta | P3 → P2 | M. Virgüez | 9-nov-2026 | Con umbrales tan flojos, un gate casi nunca falla y da una falsa sensación de seguridad. Subí la severidad porque afecta a todas las pruebas futuras, no solo a esta. |
| DEF-03 | Media → Media | P3 → P2 | M. Virgüez | 12-nov-2026 | Cumple el SLO (máximo 857 ms contra 2000 ms), por eso no es Alta. Subí la prioridad porque una de cada cinco peticiones es lenta sin explicación, y hay que entenderlo antes de fijar un umbral de P99. |

## 2.5 Hallazgos adicionales (no priorizados)

Durante la revisión del repositorio vi otros puntos que no cambian los resultados medidos, pero conviene corregirlos:

- scripts/carga.js tiene la función handleSummary exportada dos veces (líneas 61 y 68) y un import a mitad del archivo. Las pruebas sí se ejecutaron, así que probablemente es un error de la última edición. Lo confirmo con k6 inspect scripts/carga.js.

- El token `abcdef0123456789` está escrito dentro de los tres scripts. Es el token público de demo, pero sigue siendo una credencial en el código.
- `config.md` dice que `data/users.json` tiene 10 perfiles, pero el archivo tiene 8.
- La memoria RAM del equipo generador de carga estuvo entre 89 % y 94 % durante la prueba de resistencia.

No los priorizo porque no afectan las métricas, pero quedan cubiertos por el gate G1 y la acción A5.

### 2.6 Riesgos y pendientes

- QuickPizza es un servicio público de terceros. No puedo ver el servidor, así que las causas internas del Backend son hipótesis y las marco como tal.
- Cada prueba se ejecutó una sola vez. Sin repeticiones no se mide la variabilidad entre corridas.
- El consumo del generador se midió con tres capturas del Administrador de tareas, no de forma continua.

## 3. Análisis causa-raíz

Para cada defecto usé dos herramientas que se complementan. El Ishikawa abre el abanico de causas posibles (lo presento como tabla por categoría) y el 5 Whys profundiza en la cadena más probable. Cada causa lleva una marca: **Verificado** si tengo evidencia, o **Por validar** si es una suposición que todavía no puedo comprobar.

### 3.1 DEF-01. Throughput bajo el SLO

#### Ishikawa

| Categoría | Causa possible | Estado |
|---|---|---|
| Método (modelo de carga) | El think time (1-3 s y 1-4 s) hace que cada iteración dure ~4.85 s, es decir ~0.41 req/s por VU | Verificado (informe y resultados) |
| Método (modelo de carga) | Solo se usaron 20 y 15 VUs en las pruebas donde se evaluó el throughput | Verificado (`carga.js`, `resistencia.js`) |
| Proceso | El SLO de 15 req/s se definió sin calcular cuántos VUs necesita el modelo | Verificado (`config.md` frente a los scripts) |
| Medición | El throughput se calcula como promedio de toda la prueba, incluidas las rampas | Por validar |
| Entorno | El servicio público limita las peticiones por segundo | Por validar (en estrés se promedió 18.43 req/s, así que parece soportar más) |

Matías Virgüez

### 5 Whys

1. ¿Por qué el throughput fue 7.60 y 6.13 req/s? Porque cada VU genera solo ~0.41 req/s.
2. ¿Por qué cada VU genera tan pocas? Porque cada iteración dura ~4.85 s: son 2 peticiones más los tiempos de espera.
3. ¿Por qué hay tanto tiempo de espera? Porque puse un think timé realista, y con solo 20 y 15 VUs la carga total queda baja.
4. ¿Por qué no lo noté antes de ejecutar? Porque el SLO de 15 req/s no se comparó con el modelo. Hacen falta ~37 VUs (15 ÷ 0.41), no 20.
5. ¿Por qué no se comparó? Porque no existe un paso de revisión que valide que el SLO y el modelo de carga sean coherentes antes de correr la prueba.

**Causa raíz:** el SLO de throughput se definió sin validarlo contra el modelo de carga.

**Verificación:** en estrés (más VUs) el promedio llegó a 18.43 req/s. Con 45 VUs se esperan ~18 req/s, con margen sobre el SLO. Que el servicio sostenga 15 req/s de forma estable con 45 VUs queda **por validar** con una nueva ejecución.

## 3.2 DEF-02. Umbrales flojos y sin punto de quiebre

### Ishikawa

| Categoría | Causa posible | Estado |
|---|---|---|
| Método | Los umbrales de estrés (1500/3000 ms y 5 %) no son los del SLO | Verificado (estres.js) |
| Método | La rampa termina en 100 VUs y no hay un criterio de parada | Verificado (estres.js) |
| Proceso | No hay trazabilidad entre el SLO y los thresholds de cada prueba | Verificado (comparación config.md y scripts) |
| Entorno | El servicio público compartido es robusto o tiene protecciones, y por eso no se degrada | Por validar |
| Personas | Se aflojaron los umbrales para que k6 no marcara fallo en la prueba de estrés | Por validar (es una suposición mía que debo confirmar) |

### 5 Whys

1. ¿Por qué los umbrales de estrés no detectan nada? Porque son más flexibles que el SLO y el sistema nunca se acercó a ellos (máximo 857 ms y 0 % de error).
2. ¿Por qué son más flexibles? Porque en estrés se espera algo de degradación y se quiso evitar que la prueba fallara antes de terminar.

3. ¿Por qué el sistema no se degradó? Porque la rampa se detuvo en 100 VUs, y a ese nivel QuickPizza respondió igual que con 20.
4. ¿Por qué se detuvo en 100? Porque se siguió el esquema de subir de 10 en 10 hasta 100, sin definir cuándo parar ni cuándo seguir subiendo.
5. ¿Por qué faltó ese criterio? Porque el diseño de la prueba no relacionó los umbrales con el SLO ni definió una condición de salida.

**Causa raíz:** el diseño de la prueba de estrés no tiene trazabilidad con el SLO ni un criterio de parada.

**Verificación:** los umbrales y los máximos observados se pueden comprobar en `estres.js` y `results/estres.csv`. Si el servicio realmente tiene protecciones o si se degrada con más carga, queda **por validar** ampliando la rampa con cuidado.

### 3.3 DEF-03. Cola larga de latencia en POST /api/pizza

#### Ishikawa

| Categoría | Causa posible | Estado |
|---|---|---|
| Entorno (backend) | Cierto tipo de petición dispara un trabajo más pesado en el servidor | Por validar |
| Datos | Algunos perfiles (calorías o vegetariano) tardan más que otros | Por validar |
| Medición | El script no etiqueta las peticiones por perfil, así que no se puede comparar | Verificado (carga.js) |
| Medición | Solo se mira el P95 agregado y no hay métricas del servidor | Verificado (informe y dashboard) |
| Herramienta | El generador de carga estuvo con RAM entre 89 % y 94 %, lo que podría sumar ruido | Por validar (la latencia no cambia con la carga, pero conviene descartarlo) |
| Entorno (carga) | La cola aparece por saturación del servicio | Descartada: el P95 casi no cambia entre 15, 20 y 100 VUs (660, 672 y 687 ms) |

#### 5 Whys

1. ¿Por qué ~20 % de las peticiones POST superan 500 ms si la mediana es ~202 ms? Porque los tiempos forman dos grupos: uno de 100-200 ms y otro de 500-700 ms.
2. ¿Por qué hay dos grupos? Suposición: algunas peticiones activan un cálculo más pesado en el servidor, por ejemplo según el perfil.

3. ¿Por qué no depende de la carga? Porque el P95 se mantiene entre 660 y 687 ms con 15, 20 y 100 VUs. Eso apunta a un costo fijo por petición y no a una cola por saturación.
4. ¿Por qué no pude confirmarlo? Porque el script no etiqueta las peticiones por perfil y no tengo métricas del servidor.
5. ¿Por qué falta esa información? Porque la observabilidad se limitó al P95 agregado y a tres capturas manuales del equipo cliente.

**Causa raíz:** observabilidad insuficiente para explicar la latencia (solo P95 agregado y sin etiquetas por perfil). El origen exacto dentro del backend sigue **por validar**.

**Verificación:** los conteos de peticiones lentas y el histograma salen de los CSV. Para validar la hipótesis del perfil, la acción A3 etiqueta cada petición con su perfil y compara el P95 por perfil.

## 4. Plan de mejora

Las acciones salen de las causas raíz, no de los síntomas. Las ordené por impacto frente a esfuerzo: primero lo que ayuda mucho y cuesta poco.

| # | Acción | Tipo | Defecto | Impacto | Esfuerzo | Responsable | Fecha | KPI / umbral de verificación |
|---|---|---|---|---|---|---|---|---|
| A1 | Subir la carga a 45 VUs (mínimo calculado: 37) y validar el SLO de throughput con la fórmula VUs × 0.41 | Corrección | DEF-01 | Alto | Bajo | M. Virgüez | 4-nov | Throughput ≥ 15 req/s en la fase estable de la prueba de carga |
| A2 | Igualar los thresholds de estrés al SLO (800 ms, 2000 ms, 1 %) y activar `abortOnFail` | Corrección | DEF-02 | Alto | Bajo | M. Virgüez | 4-nov | 100 % de los scripts con thresholds iguales al SLO |
| A3 | Etiquetar cada petición con su perfil y comparar el P95 por perfil | Pruebas adicionales | DEF-03 | Medio | Bajo | M. Virgüez | 6-nov | P95 por perfil reportado; se identifica si un perfil concentra las |

| # | Acción | Tipo | Defecto | Impacto | Esfuerzo | Responsable | Fecha | KPI / umbral de verificación |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | peticiones lentas |
| A4 | Agregar una métrica propia de peticiones > 500 ms y un threshold de P99 | Prevención | DEF-03 | Medio | Bajo | M. Virgüez | 6-nov | P99 POST < 1000 ms y peticiones > 500 ms ≤ 25 % |
| A5 | Limpiar los scripts: quitar el `handleSummary` duplicado, mover el token a una variable de entorno, corregir `config.md` (8 perfiles) y liberar RAM del generador | Corrección | Hallazgos adicionales | Medio | Bajo | M. Virgüez | 5-nov | `k6 inspect` sin errores, 0 secretos en el repositorio, RAM del generador < 80 % |
| A6 | Ampliar la rampa de estrés (120 y 150 VUs) con criterio de parada, de forma moderada porque el servicio es público | Pruebas adicionales | DEF-02 | Alto | Medio | M. Virgüez | 9-nov | Punto de quiebre documentado (VUs y req/s), o registro de que no hubo quiebre hasta N VUs |
| A7 | Enviar las métricas de k6 y del generador a Grafana de forma continua en vez de usar | Infraestructura | DEF-03 | Medio | Medio | M. Virgüez | 12-nov | 100 % de las pruebas con métricas continuas |

| # | Acción | Tipo | Defecto | Impacto | Esfuerzo | Responsable | Fecha | KPI / umbral de verificación |
|---|---|---|---|---|---|---|---|---|
| | capturas | | | | | | | |

## 5. Propuesta técnica de quality gates en CI/CD

### 5.1 Objetivo

Evitar que los tres defectos vuelvan a aparecer sin que nadie se dé cuenta. La idea es simple: si una regla se puede medir, se automatiza, y si falla, el pipeline se detiene (fail-fast) antes de que el problema llegue más lejos.

### 5.2 Diseño del pipeline

1. **Etapa 1, validación (en cada commit o PR, ~1 min):** revisar que los scripts sean válidos y que no tengan secretos (gate G1).
2. **Etapa 2, smoke de rendimiento (en cada PR, ~2 min):** 5 VUs durante 1 minuto (gates G2, G3 y G4).
3. **Etapa 3, carga completa (nocturna o antes de un release, ~6 min):** 45 VUs con el modelo real (gates G2 a G6).
4. **Etapa 4, estrés y resistencia (semanal o manual):** con criterio de parada (gates G7 y G8).
5. **Etapa 5, reporte:** publicar los resultados en Grafana para ver la tendencia.

Si falla la etapa 1, no se ejecutan las siguientes. Así no se gastan minutos ni se carga un servicio público con un script roto.

### 5.3 Gates y umbrales

Uso dos niveles. La **alerta** avisa pero no bloquea, y está en la línea base medida más ~25 % (por ejemplo, P95 de home ~105 ms y P95 de pizza ~660-687 ms). El **bloqueo** detiene el pipeline y coincide con el SLO.

| Gate | Qué mide | Alerta (no bloquea) | Bloqueo (fail-fast) | Defecto que previene |
|---|---|---|---|---|
| G1. Validez y seguridad del script | k6 inspect sin errores; 0 secretos; calidad del código | Duplicación > 3 % | Cualquier error de sintaxis, un secreto o una vulnerabilidad nueva | Hallazgos adicionales |
| G2. Latencia GET / | P95 | > 130 ms | ≥ 800 ms | DEF-02 |
| G3. Latencia POST /api/pizza | P95 | > 850 ms | ≥ 2000 ms | DEF-02 |

| Gate | Qué mide | Alerta (no bloquea) | Bloqueo (fail-fast) | Defecto que previene |
|---|---|---|---|---|
| G4. Error y checks | Tasa de error y tasa de checks | Error > 0.5 % | Error ≥ 1 % o checks < 99 % | DEF-02 |
| G5. Throughput en carga | req/s en la fase estable | < 17 req/s | < 15 req/s | DEF-01 |
| G6. Cola larga del POST | P99 y % de peticiones > 500 ms | P99 > 900 ms o % > 25 | P99 ≥ 1000 ms o % > 30 | DEF-03 |
| G7. Estrés con criterio de parada | P95 y error durante la rampa | Aviso al llegar a 120 VUs | La prueba se aborta si P95 ≥ SLO o error ≥ 1 %, y se registra el punto de quiebre | DEF-02 |
| G8. Salud del generador | RAM del equipo que genera la carga | > 80 % | Solo alerta (no bloquea) | DEF-03 |

### 5.4 Fragmento de k6 con los umbrales

```
import http from 'k6/http';
import { Rate } from 'k6/metrics';

const slow_pizza = new Rate('slow_pizza'); // peticiones POST > 500 ms

export const options = {
  thresholds: {
    'http_req_duration{endpoint:home}': ['p(95)<800'],
    'http_req_duration{endpoint:pizza}': ['p(95)<2000', 'p(99)<1000'],
    http_req_failed: [{ threshold: 'rate<0.01', abortOnFail: true, delayAbortEval:
'30s' }],
    checks: ['rate>0.99'],
    http_reqs: ['rate>=15'],
    slow_pizza: ['rate<0.30'],
  },
};

// dentro de la función principal, después del POST:
// slow_pizza.add(res.timings.duration > 500);
```

Cuando un threshold falla, k6 termina con código de salida distinto de cero (99). Eso es lo que hace que el job del pipeline falle sin trabajo extra.

### 5.5 Fragmento de GitHub Actions

```
name: quality-gate-rendimiento
on:
  pull_request:
  schedule:
    - cron: '0 6 * * 1'   # carga completa los lunes
```

```
jobs:
  validar-scripts:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gitleaks/gitleaks-action@v2
      - uses: grafana/setup-k6-action@v1
      - run: for f in scripts/*.js; do k6 inspect "$f" > /dev/null; done

  smoke-rendimiento:
    needs: validar-scripts
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: grafana/setup-k6-action@v1
      - run: k6 run scripts/smoke.js   # 5 VUs, 1 min, mismos thresholds
```

Para que funcione hay que crear `scripts/smoke.js` (una copia reducida de `carga.js`) y ajustar las versiones de las acciones a las vigentes.

### 5.6 Herramientas y justificación

| Herramienta | Para qué | Por qué la elegí |
|---|---|---|
| k6 (thresholds) | Pruebas de rendimiento y gates de latencia, error y throughput | Ya lo usé en las tres pruebas, y sus thresholds hacen fallar el proceso solos (fail-fast) |
| GitHub Actions | Orquestar el pipeline | El repositorio ya está en GitHub y no requiere infraestructura extra |
| Gitleaks | Detectar secretos en el código | Es gratuito, rápido y se integra en un paso del pipeline |
| SonarQube o SonarCloud | Quality gate de código (vulnerabilidades, duplicación) | Permite fijar condiciones sobre el código nuevo, y se ve en el material del curso |
| Grafana | Ver tendencias entre ejecuciones | Ya construí un dashboard de KPIs y sirve para detectar degradación gradual |

### 5.7 Política fail-fast y excepciones

- Un gate en bloqueo detiene el pipeline y no se puede hacer merge ni release.
- Si un gate de latencia falla, se repite una vez y se toma la mediana de las corridas, para descartar ruido de red.
- Una excepción solo se acepta con una justificación escrita (qué falló, por qué se acepta y hasta cuándo) y debe quedar en el historial del repositorio.
- Las alertas no bloquean, pero si una alerta se repite tres veces seguidas se trata como defecto nuevo.

### 5.8 Trazabilidad: defecto, causa raíz, acción, gate y KPI

| Defecto | Causa raíz | Acción | Gate | KPI |
|---|---|---|---|---|
| DEF-01 | SLO de throughput sin validar contra el modelo de carga | A1 | G5 | Throughput ≥ 15 req/s |
| DEF-02 | Umbrales sin trazabilidad al SLO y sin criterio de parada | A2, A6 | G2, G3, G4, G7 | Thresholds iguales al SLO; punto de quiebre documentado |
| DEF-03 | Observabilidad insuficiente (solo P95 agregado) | A3, A4, A7 | G6, G8 | P99 < 1000 ms; peticiones > 500 ms ≤ 25 % |
| Hallazgos adicionales | Falta de revisión automática de los scripts | A5 | G1 | 0 secretos; k6 inspect sin errores |

## 5.9 Riesgos y límites

- QuickPizza es un servicio público de terceros: las pruebas de estrés no deben correr en cada PR para no sobrecargarlo.
- La latencia desde un runner de GitHub puede ser distinta a la medida desde mi equipo en Colombia. Por eso los umbrales de alerta se recalibran con las primeras corridas del pipeline.
- El gate de throughput usa el promedio de la prueba, que incluye las rampas. Hay que medirlo sobre la fase estable o dejar margen (45 VUs en vez de 37).

## 6. Conclusiones

La prueba mostró un sistema estable: sin errores y con latencias muy por debajo del SLO. Aun así, encontré tres defectos reales con evidencia. El más importante es el throughput, que no cumplió el SLO porque el objetivo se definió sin calcularlo contra el modelo de carga. Los otros dos tienen la misma raíz de fondo, la falta de trazabilidad y de observabilidad: umbrales de estrés que no detectan nada, y una cola de latencia que no puedo explicar.

Por eso el plan se centra en prevenir y no solo en corregir. Alinear los umbrales con el SLO, etiquetar las peticiones, medir de forma continua y automatizar todo en un pipeline con gates hace que cada defecto tenga una regla que lo vigile. Lo que queda por validar, como el origen de la cola larga en el backend, está marcado de forma explícita para no presentar suposiciones como hechos.

## Anexo. Evidencias en el repositorio

- config.md: modelo de carga y SLO.
- scripts/carga.js, scripts/estres.js, scripts/resistencia.js: scripts de k6 y thresholds.
- results/*.csv y results/*.json: resultados crudos de las tres pruebas.
- dashboard/resultados_pruebas.csv y dashboard/observabilidad_soak.csv: resumen de KPIs y consumo del generador.
- report/Informe_tecnico_prueba_rendimiento.pdf y report/screenshots/: informe técnico y capturas.

## Referencias


- American Society for Quality. (s. f.). Five whys and five hows. https://asq.org/quality-resources/5-whys
- Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (Eds.). (2018). The site reliability workbook: Practical ways to implement SRE (cap. 10, Postmortem culture: Learning from failure). O'Reilly Media. https://sre.google/workbook/postmortem-culture/
- GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/en/actions
- Gitleaks. (s. f.). gitleaks-action [Software]. GitHub. https://github.com/gitleaks/gitleaks-action
- Goyal, A. (2024, 16 de julio). Performance testing with Grafana k6 and GitHub Actions. Grafana Labs. https://grafana.com/blog/performance-testing-with-grafana-k6-and-github-actions/
- Grafana Labs. (s. f.-a). Grafana documentation. https://grafana.com/docs/grafana/latest/
- Grafana Labs. (s. f.-b). QuickPizza [Aplicación web de demostración]. https://quickpizza.grafana.com
- Grafana Labs. (s. f.-c). Thresholds. En Grafana k6 documentation. https://grafana.com/docs/k6/latest/using-k6/thresholds/
- Improvement Cymru Academy. (s. f.-a). The 5 whys: A tool for root cause analysis [Toolkit guide]. Public Health Wales NHS Trust. https://performanceandimprovement.nhs.wales/functions/quality-safety-and-improvement/improvement/improvement-cymru-academy/resource-library/academy-toolkit-guides/the-5-whys-toolkit-guide/
- Improvement Cymru Academy. (s. f.-b). Fishbone toolkit guide [Toolkit guide]. Public Health Wales NHS Trust. https://executive.nhs.wales/functions/quality-safety-and-improvement/improvement-cymru/improvement-cymru-academy/resource-library/academy-toolkit-guides/fishbone-toolkit-guide/
- International Software Testing Qualifications Board. (2024). Certified Tester Advanced Level Test Management syllabus (Versión 3.0). https://www.istqb.org/wp-content/uploads/2024/11/ISTQB_CTAL-TM_Syllabus_v3.0_zKjKsaN.pdf
- Lunney, J., & Lueder, S. (2016). Postmortem culture: Learning from failure. En B. Beyer, C. Jones, J. Petoff, & N. R. Murphy (Eds.), Site reliability engineering: How Google runs production systems (cap. 15). O'Reilly Media. https://sre.google/sre-book/postmortem-culture/
- SonarSource. (s. f.). Understanding quality gates. En SonarQube Server documentation. https://docs.sonarsource.com/sonarqube-server/quality-standards-administration/managing-quality-gates/introduction-to-quality-gates
- Universidad de La Salle. (2026). Recurso educativo digital 2: Pruebas y rendimiento de carga [Material del curso]. Calidad en Software II.

- Virgüez, M. (2026). prueba-rendimiento-quickpizza [Repositorio de código]. GitHub. https://github.com/Matiasv18/prueba-rendimiento-quickpizza
- Virgüez, M. (2026, 19 de septiembre). Script y ejecución de prueba de carga [Informe técnico]. Calidad en Software II.